Relatório 1: Bhaskara e Teclado Alfanumérico

Elisa Belquis de Assumpção
João Lucas Medina Kormann

I. Introdução

Este relatório descreve o desenvolvimento e a implementação das atividades realizadas no Laboratório 1 da disciplina INE-5411, Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck. A partir da análise por trechos do código, buscamos trazer um entendimento total não só de nossa resolução, mas também das dificuldades encontradas.

II. Desenvolvimento

A. Desenvolvimento Bhaskara

Como primeiro exercício proposto, temos a aplicação da Fórmula de Bhaskara em um código assembly, a qual foi dividida em duas etapas, uma com as variáveis a, b e c já definidas e armazenadas em espaços de memórias "word" e outra sem elas estarem inicialmente definidas, sendo então lidas pelo console através do teclado.

Tratando inicialmente dos fatores que se aplicam em ambos os casos, iniciamos delegando os registradores responsáveis por cada variável, sendo eles $t9 para a, $t8 para b e $t7 para c.

```
lw      $t9, a          # Salva o valor de A para o registrador t9
lw      $t8, b          # Salva o valor de B para o registrador t8
lw      $t7, c          # Salva o valor de C para o registrador t7
```

Figura 1- Registradores para a ,b, c

Após isso, partimos para a aplicação inicial da fórmula de bhaskara em si, calculando $\Delta = b^2 - 4ac$, separando em partes menores e usando os registradores com os valores salvos para os cálculos.

```
mul     $t0, $t8, $t8   # b^2

mul     $t1, $t9, 4     # Bloco da operação 4ac
mul     $t1, $t1, $t7

sub     $t0, $t0, $t1   # Operação b^2-4ac
```

Figura 2 - Calculo de b²-4ac

Em sequência, para a aplicação de raíz quadrada, tivemos uma dificuldade inicial devido a não podermos aplicar sua função com variáveis inteiras (word) porém, pesquisando um pouco a respeito, encontramos uma forma para solucionar tal dificuldade, passamos o valor para um registrador de ponto flutuante ($f0) e então convertemos o valor de word para float. Dessa forma, foi possível executar o comando *sqrt.s* que calcula a raíz quadrada do valor no registrador selecionado e, após isso, retornamos o valor para word e devolvemos ele para o registrador de origem ($t0).

```
mtc1 $t0, $f1           # Move o inteiro de $t0 para o registrador de ponto flutuante $f4
cvt.s.w $f1, $f1        # Converte de inteiro (word) para float (single)

sqrt.s  $f1, $f1        # Faz a raiz quadrada de delta

cvt.w.s $f1, $f1        # Converte para word novamente
mfc1  $t0, $f1          # Move do registrador de ponto flutuante f1 para t0
```

Figura 3- Cálculo raíz quadrada de delta

Seguindo para a aplicação da Fórmula de Bhaskara em si: $Bhaskara = \frac{-b \pm \sqrt{\Delta}}{2a}$, explicaremos utilizando o mesmo formato do cálculo anterior, separando-o em etapas menores e executando os cálculos com os valores dos registradores.

```
sub     $t1, $zero, $t8 # Operação para obter -b

mul     $t2, $t9, 2     # Operação 2a

add     $t3, $t1, $t0   # Operação (-b)+ raiz quadrada de delta
div     $s0, $t3, $t2   # Divide a operação acima por 2a

sub     $t3, $t1, $t0   # Operação (-b) - raiz quadrada de delta
div     $s1, $t3, $t2   # Divide a operação acima por 2a
```

Figura 4 - Cálculo Bhaskara

Por fim, carregamos o valor 1 no registrador $v0 para, com o *syscall*, poder imprimir os valores inteiros (word) obtidos pela nossa aplicação da fórmula, salvando também um endereço x1 e x2 para uso futuro. Vale notar que intercalando entre a exibição dos 2 valores, optamos por inserir o valor 4 no registrador $v0 e imprimir a string " " apenas para servir como uma forma de separar as duas raízes impressas, x1 e x2.

```
li      $v0, 1          # Carrega 1 em v0 para printar inteiro
add     $a0, $a0, $s0   # Salva em a0 o valor final da conta
sw      $a0, x1         # Salva em x1 o valor final da conta
syscall

li      $v0, 4          # Carrega 4 em v0 para printar uma string
la      $a0, espaco     # Salva em a0 um espaço em branco (" ") só para separar os dois valores de x na hora da exibicao
syscall

li      $v0, 1          # Carrega 1 em v0 para printar inteiro
add     $a0, $zero, $s1 # Salva em a0 o valor final da conta
sw      $a0, x2         # Salva em x2 o valor final da conta
syscall
```

Figura 5 - Exibição dos resultados no console

Exemplo de impressão com a = 1, b = -5 e c = 6:

Figura 6 - Console Bháskara

Nesta versão foram usadas 43 linhas de código contidas na coluna Basic e 29 linhas na coluna Source. Isso acontece porque o montador expande as pseudo-instruções em instruções MIPS básicas. A coluna Basic, mostra as instruções reais que serão enviadas. Já a coluna Basic mantém o código mais abstrato, que foi o que escrevemos.

A. 1 Bhaskara com leitor de teclado

Como segunda parte, temos a adaptação do código para ler um valor digitado pelo teclado. Para tal, não houve muitas dificuldades, bastou alterar o ínicio do nosso código. Onde anteriormente direcionamos diretamente o valor das words para os registradores, agora temos um passo intermédio, onde armazenamos o imediato 5 em $v0, para ler uma word do teclado e salvamos o valor adquirido no próprio $v0 para os registradores da mesma forma como delegados anteriormente, sendo o primeiro (a) para $t9, o segundo (b) para $t8 e o terceiro (c) para $t7. Após isto, o código segue da mesma maneira, possuindo a mesma saída já apresentada.

```
li      $v0, 5              # Carrega 5 em v0 para capturar um valor de input
syscall
move    $t9, $v0            # Salva o valor de A em t9

li      $v0, 5              # Carrega 5 em v0 para capturar um valor de input
syscall                     #Valor de B
move    $t8, $v0            # Salva o valor de B em t8

li      $v0, 5              # Carrega 5 em v0 para capturar um valor de input
syscall
move    $t7, $v0            # Salva o valor de C em t7
```

Figura 7 - Captação de valores pelo teclado

Assim como na versão anterior, foram usadas mais linhas linhas de código contidas na coluna Basic (46) em relação à coluna Source (35).

B. Desenvolvimento Digital Lab Sim

Para o segundo exercício, separamos completamente o desenvolvimento de suas partes pois acreditamos que assim seria a melhor forma de entendê-lo e aplicá-lo. Porém, como obstáculo inicial em ambos, houve um certo desafio no entendimento do funcionamento do Digital Lab Sim e também de como encontrar/usar os valores de endereço disponíveis. Após ler o material contido na opção *help* e pesquisar sobre o funcionamento da ferramenta, conseguimos entender como aplicá-lo.

```
This tool is composed of 3 parts : two seven-segment displays, an hexadecimal keyboard and counter
Seven segment display
Byte value at address 0xFFFF0010 : command right seven segment display
Byte value at address 0xFFFF0011 : command left seven segment display
Each bit of these two bytes are connected to segments (bit 0 for a segment, 1 for b segment and 7 for point
```

Figura 8 - *Help* do Digital Lab Sim

B.1 Imprimindo números no display

Como primeira proposta do segundo exercício, temos a exibição dos números de 0 a 9 em um display do Digital Lab Sim. Para isso, inicialmente salvamos a word 0xFFFF0010 que refere ao endereço o qual o simulador enxerga como byte para a exibição do led direito.

```
.data
led_direita:  .word 0xFFFF0010  # Valor do endereço referente ao led direito do digital lab sim
```

Figura 9 - Criação word do endereço do led

Após isso, salvamos o valor da word em um registrador $t1, carregamos o valor 1000 em $a0 e o valor 32 em $v0 para assim criar um atraso de 1 segundo entre a exibição de cada número no display led.

```
lw  $t1, led_direita        # Salva em t1 o valor do endereço referente ao led direito do digital lab sim
li  $a0, 1000               # Tempo em milissegundos referente a 1 segundo
li  $v0, 32                 # Carrega 32 no registrador v0 para poder contar 1 segundo após o syscall e continuar o codigo
```

Figura 10 - Carregamento de informações

Por fim, carregamos em $t0 o valor do byte referente a cada número em um display de 7 segmentos e usamos o comando **sb** (store byte) para carregar esse valor salvo em $t0 no endereço 0xFFFF0010 (salvo em $t1), intercalando com syscall para chamar o atraso de 1 segundo.

```
li  $t0, 0x3F               # Carrega o byte referente a 0 no display 7 segmentos
sb  $t0, 0($t1)             # Salva esse byte no endereço referenciado por t1
syscall                     # Atraso de 1 seg

li  $t0, 0x06               # Carrega o byte referente a 1 no display 7 segmentos
sb  $t0, 0($t1)             # Salva esse byte no endereço referenciado por t1
syscall                     # Atraso de 1 seg
```

Figura 11 - Código para exibição do número

Com isso, obtemos uma sequência que conta automaticamente de 0 a 9, gastando um total de 9 segundos para que seja possível visualizar todos os números e então finalizamos carregando 10 em $v0 e utilizando o syscall para encerrar o sistema.

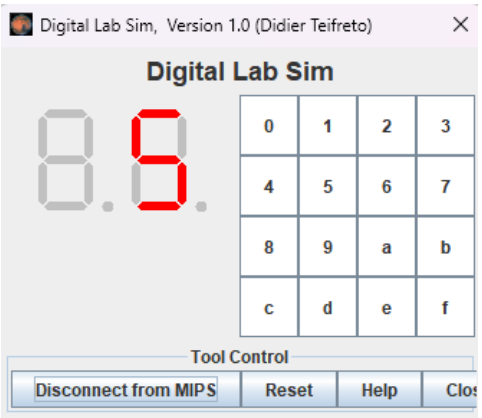

Figura 12 - Exibição do valor 5 no display

B.2 Imprimindo os números do teclado no display

Como última proposta, temos a impressão do número selecionado pelo teclado alfanumérico. Este por sua vez, foi o mais desafiador, principalmente para entendermos o funcionamento do teclado e como sua leitura era executada. Assim como no caso anterior, separamos em words os valores referentes aos endereços, porém desta vez salvamos também 0xFFFF0012 para referenciar a linha a ser ativa para leitura e 0xFFFF0014 para referenciar a tecla da linha.

```
.data
linha:              .word 0xFFFF0012        # Endereço relativo à linha do teclado
leitura_teclado:    .word 0xFFFF0014        # Endereço relativo à leitura da tecla
led_direita:        .word 0xFFFF0010        # Valor do endereço referente ao led direito do digital lab sim
```

Figura 13 - Delegação dos endereços

Além disso, criamos também duas tabelas que serão utilizadas, uma referente ao código retornado quando selecionamos um valor do teclado alfanumérico e outra referente à sua "tradução" em byte para o display de 7 segmentos, onde ambas estão ordenadas da mesma forma para que o deslocamento de uma seja referente ao mesmo deslocamento da outra.

```
# Tabela com os valores dos numeros no display de 7 segmentos
tabela_segmentos:       .byte 0x3F,0x06,0x5B,0x4F
                        .byte 0x66,0x6D,0x7D,0x07
                        .byte 0x7F,0x6F,0x77,0x7C
                        .byte 0x39,0x5E,0x79,0x71

# Tabela dos códigos do teclado do digital lab sim
tabela_codigos:         .byte 0x11,0x21,0x41,0x81
                        .byte 0x12,0x22,0x42,0x82
                        .byte 0x14,0x24,0x44,0x84
                        .byte 0x18,0x28,0x48,0x88
```

Figura 14 - Tabelas de bytes

Partindo para os comandos em si, separamos o funcionamento em vários blocos para efetuarmos laços quando necessário. Assim como nos casos anteriores, iniciamos guardando as informações de endereços nos registradores $t0, $t1, $t2 e iniciamos o registrador $t3 com 1 para servir de contador futuramente.

```
main:                                   # Inicio do programa
        lw      $t0, linha              # Carrega o endereço da ativação da linha no registrador t0
        lw      $t1, leitura_teclado    # Carrega o endereço relativo à leitura no registrador t1
        lw      $t2, led_direita        # Carrega o endereço relativo ao led direito no registrador t2
        li      $t3, 1                  # Carrega 0001 em t3 para ser usado futuramente
```

Figura 15 - Inicialização registradores

Em seguida, partimos para o bloco “lendo_teclado” que tem como funcionamento básico a ativação de 1 bit por vez do byte das linhas para ativar a linha a ser lida, executando isto em um total de 4 vezes para assim ativar 1 linha de cada vez e checar (comparando $t4 com $zero), se algum valor foi pressionado durante este processo.

```
lendo_teclado:
        sb      $t3, 0($t0)             # Ativa a linha, carregando o valor de t3 no endereço de ativação da linha
        lb      $t4, 0($t1)             # Lê a tecla e armazena em t4
        bne     $t4, $zero, pressionada # Compara t4 com zero para saber se alguma tecla foi pressionada, se foi, passa para o proximo bloco
        sll     $t3, $t3, 1             # Da shift left em t3 para assim ativar a proxima linha, exemplo 0001 (linha 1) -> 0010 (linha 2)
        ble     $t3, 8, lendo_teclado   # Enquanto t3 for menor ou igual a 8(ou seja, enquanto ainda nao tiver passado por todas as linhas), mantem o loop
        j       fim                     # Nenhuma tecla pressionada até o final, jogando assim para o fim do programa
```

Figura 16 - bloco lendo_teclado

Caso seja identificado que alguma tecla tenha sido pressionada, o programa “pula” para “pressionada” e carrega o valor 0 em $t3 a fim de ser utilizado como contador. Então, a pseudo-instrução **la** carrega o endereço de “tabela_codigos” em $t5 que posteriormente irá comparar a tabela de códigos do teclado com o valor que foi pressionado.

```
pressionada:
        li      $t3, 0                  # Carrega 0 em t3 para servir de contador futuramente
        la      $t5, tabela_codigos     # Carrega a tabela dos codigos do teclado para comparar com o resultado adquirido
```

Figura 17 - pressionada

Inicia-se, então o bloco “traduzir_tecla” que tem como princípio descobrir qual tecla foi pressionada, comparando seus endereços. É parecido com um for que, ao completar 16 iterações sem sair do loop (percorreu todas as teclas), segue para o fim do programa.

```
traduzir_tecla:
        lb      $t6, 0($t5)             # Carrega em t6 o byte referente a t5 (no primeiro caso, o byte 0x11)
        beq     $t6, $t4, iguais        # Verifica se o byte de t6 é igual ao byte t4(byte salvo pelo teclado), se for, passa para o proximo bloco
        addi    $t3, $t3, 1             # Adiciona 1 ao contador
        addi    $t5, $t5, 1             # Adiciona 1 a t5 para checar o proximo byte ( em primeira instacia, após verificar que 0x11 nao é o byte
                                        #desejado, passa para 0x21)
        blt     $t3, 16, traduzir_tecla #Fica nesse loop até o contador resultar em 16(passou por todas as teclas)
        j       fim                     #Não achou tecla pressionada, jogando assim para o fim do programa
```

Figura 18 - bloco traduzir_tecla

Quando a instrução **beq** identifica que o endereço de alguma tecla da tabela é igual ao selecionado, o deslocamento armazenado em $t3 é aplicado na tabela dos números traduzidos, descobrindo sua referência em 7 segmentos. Este byte é carregado em $t8 e o display exibe o valor selecionado no teclado.

```
iguais:
        la      $t7, tabela_segmentos   # Carrega em t7 a tabela dos numeros traduzidos em seus segmentos
        add     $t7, $t7, $t3           # Soma t7 com o contador (t3) para assim ter o deslocamento referente a qual byte foi equivalente na tabela anterior
        lb      $t8, 0($t7)             # Carrega este byte em t8

display:
        sb      $t8, 0($t2)             # Armazena no endereço relativo ao led direito o valor do byte carregado em t8
```

Figura 19 - sincronização das tabelas e carregamento do display

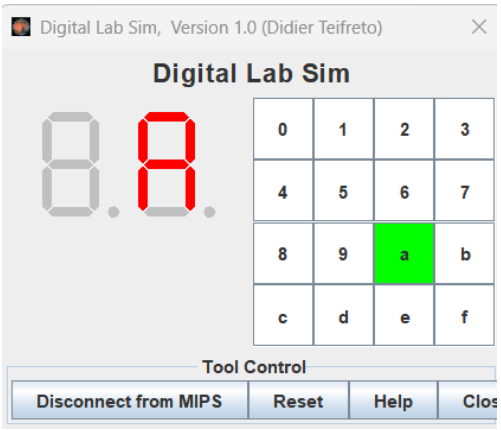

Figura 20 - Exibição do valor selecionado no teclado

O programa é encerrado quando a chamada do sistema é realizada com o valor 10 sendo armazenado em $v0. Este trecho do código é visitado de duas maneiras, sequencialmente, após o display mostrar o valor selecionado, ou caso nenhuma tecla tenha sido selecionada.

```
fim:
        li      $v0,10                  # Carrega 10 em v0 para encerrar o sistema
        syscall                         # Chama o encerramento do sistema
```

Figura 21 - Encerramento do programa